

Lộ trình phát triển hệ thống cho đề tài KLTN không đơn thuần là "code một trang web bán hàng hay quản lý", mà là **xây dựng một Công cụ Thực nghiệm HCI (Experimental Testing Web App)**.

Mục tiêu tối thượng của ứng dụng này là: **Thu thập dữ liệu chính xác, không gián đoạn và kiểm soát tốt góc nhìn của người dùng ở 3 dạng giao diện khác nhau.**



1. Mẹo Kỹ thuật Tối quan trọng cho Dự án Nghiên cứu

Trước khi chọn Tech Stack, bạn cần nắm một "bí quyết" trong lập trình web thực nghiệm HCI: **Đừng gọi mô hình ML / SHAP trực tiếp trên Backend khi người dùng đang bấm làm bài test!**

- **Tại sao?** 20 kịch bản của bạn (bao gồm 4 câu gài bẫy và 2 câu Attention Check) đã được cố ý thiết kế với các thông số AI và SHAP Value định sẵn để kiểm soát thí nghiệm.
- **Cách làm chuẩn:**
 1. **Trên Jupyter Notebook:** Bạn vẫn huấn luyện mô hình Machine Learning (Random Forest) trên tập dữ liệu thật và chạy thư viện **shap** để vẽ biểu đồ (phục vụ viết Chương 3 trong báo cáo KLTN).
 2. **Trên Web App Thực nghiệm:** Backend chỉ cần nạp (load) file JSON chứa 20 kịch bản đã pre-compute sẵn thông số AI và SHAP Value. Điều này giúp Web App chạy siêu nhanh (dưới 10ms/câu), giao diện không bị giật lag khi 30–50 người cùng làm test một lúc.



2. Tech Stack Gọn nhẹ & Dễ Kiểm soát

Nên ưu tiên một Tech Stack cực kỳ quen thuộc, dễ làm và dễ kiểm soát giao diện:

- **Backend: Python (FastAPI)**
 - *Lý do:* FastAPI nhẹ, khởi tạo nhanh, có sẵn tích hợp Swagger UI, xử lý bất đồng bộ tốt và viết cùng ngôn ngữ Python với mô hình ML.
- **Frontend: React.js + TailwindCSS + Recharts (hoặc Chart.js)**
 - *Lý do:* React giúp quản lý trạng thái (State) rất tốt khi chuyển câu hỏi, hiển thị đồng hồ đếm giờ (Timer) và render các dạng giao diện A/B/C linh hoạt. Recharts/Chart.js giúp vẽ biểu đồ cột SHAP đẹp mắt.
- **Database / Logging: SQLite hoặc PostgreSQL / Firebase**
 - *Lý do:* Chỉ cần lưu dữ liệu câu trả lời, thời gian ra quyết định (tính bằng mili-giây) và kết quả bài khảo sát NASA-TLX. SQLite là quá đủ và không cần cài đặt server rườm rà.



3. Kiến trúc Luồng Người dùng (User Flow) & 3 Nhóm Giao diện

Trang web sẽ hoạt động theo luồng 4 màn hình tuần tự:

[MÀN HÌNH 1: HƯỚNG DẪN & NHẬP THÔNG TIN]

| (Người dùng điền Tên/Mã SV, hệ thống tự động ngẫu nhiên phân vào Nhóm A, B, hoặc C)



[MÀN HÌNH 2: LÀM TEST 20 HỒ SƠ]

| (Hiển thị 1 trong 3 dạng giao diện; Ghi nhận thời gian đếm giây và lựa chọn)



[MÀN HÌNH 3: KHẢO SÁT NASA-TLX]

| (5-6 câu hỏi trắc nghiệm đánh giá mức độ mệt mỏi trí óc sau khi làm test)



[MÀN HÌNH 4: CẢM ƠN & KẾT THÚC]

Chi tiết thiết kế 3 dạng Giao diện (A/B/C) trên Màn hình 2:

- Giao diện A (Nhóm Control - Black-box):**
 - Bên trái: Bảng thông tin 7 thuộc tính hồ sơ.
 - Bên phải: Một thẻ (Card) hiển thị duy nhất: "**Dự đoán AI: DUYỆT (Độ tin cậy: 92%)**".
 - Bên dưới: 2 nút bấm [**Đồng ý với AI**] và [**Bác bỏ AI / Thay đổi quyết định**].
- Giao diện B (Nhóm Treatment 1 - Standard XAI):**
 - Giống Giao diện A, nhưng có thêm:
 - Biểu đồ cột ngang (Diverging Bar Chart) hiển thị 3–4 yếu tố SHAP quan trọng nhất (thanh xanh là điểm cộng, thanh đỏ là điểm trừ).
 - 1 dòng chữ gợi ý phản thực tế (Counterfactual): "*Nếu thêm niên tăng từ 0.8 năm lên 1.0 năm, hệ thống sẽ Duyệt*".
- Giao diện C (Nhóm Treatment 2 - Over-complex XAI):**
 - Giống Giao diện B, nhưng hiển thị toàn bộ Biểu đồ SHAP Force Plot gốc phức tạp, kèm ma trận các chỉ số toán học đầy đủ để cố tình châm ngòi cho sự quá tải nhận thức.



4. Plan Phát triển Web App (Chia làm 4 Sprint)

- ♦ **Sprint 1: Chuẩn bị Backend & Data Schema (3–4 ngày)**
 - Khởi tạo dự án FastAPI.
 - Tạo file `scenarios.json` chứa 20 kịch bản đã chuẩn hóa.
 - Thiết kế Database Schema để lưu kết quả log:
 - `User`: `user_id`, `group_assigned` (A/B/C), `start_time`, `end_time`.
 - `ResponseLog`: `user_id`, `scenario_id`, `user_decision` (Đồng ý/Bác bỏ), `time_spent_seconds`, `is_correct_on_error_case`.
 - `SurveyLog`: Điểm NASA-TLX (thang điểm 1–7).
- ♦ **Sprint 2: Xây dựng Frontend & Lập trình 3 Giao diện (5–7 ngày)**
 - Dựng Layout chuẩn cho Web App bằng React & TailwindCSS.
 - Tạo thuật toán **Randomize Group** (Phân bổ đều người dùng vào 3 nhóm A, B, C khi họ truy cập).
 - Viết component vẽ biểu đồ SHAP bằng Recharts (cho Nhóm B và C).
 - Gắn đồng hồ đếm thời gian ngằm (Stopwatch/Timer) bắt đầu đếm khi câu hỏi xuất hiện và dừng lại khi người dùng bấm nút chọn.
- ♦ **Sprint 3: Tích hợp API, Lưu Log & Bảng khảo sát NASA-TLX (3–4 ngày)**
 - Kết nối Frontend với Backend API để lưu dữ liệu realtime sau mỗi câu bấm.
 - Dựng Màn hình 3 (Bảng khảo sát NASA-TLX ngắn dạng Slider 1–7 điểm).
 - Làm trang Admin ẩn (`/admin/export`) giúp bạn bấm một nút là tải toàn bộ dữ liệu người dùng về thành file **CSV/Excel** để sẵn sàng phân tích thống kê.
- ♦ **Sprint 4: Kiểm thử Nội bộ (Internal Testing) & Deploy (2–3 ngày)**
 - Chạy thử nghiệm nội bộ với 2–3 người bạn để kiểm tra các lỗi giao diện, đếm giờ sai, hoặc đứt kết nối Database.

- Deploy ứng dụng lên các nền tảng miễn phí/chi phí thấp:
 - Frontend: **Vercel** hoặc **Netlify**.
 - Backend: **Render** hoặc **Railway**.
 - Database: SQLite (Lưu file trực tiếp trên server) hoặc Supabase/PostgreSQL.

Kế hoạch này giúp bạn hoàn thành toàn bộ sản phẩm Web App phục vụ nghiên cứu trong khoảng **2 đến 3 tuần làm việc**.

Bạn thấy cấu trúc luồng người dùng và 4 Sprint phát triển này đã đủ rõ ràng để bắt tay vào viết code chưa?